

helixqa_integration_sec01

1. Phase 0: Constitution & Governance Update

Before any code changes are merged, the governance layer that constrains every subsequent decision must be made consistent across the HelixCode enterprise system and its submodules. This phase addresses the missing Article XI §11.9 User-Mandate Forensic Anchor in HelixCode’s three primary governance files — CONSTITUTION.md, CLAUDE.md, and AGENTS.md — and aligns the anti-bluff constant identifier from CONST-017 to CONST-035 for cross-repository parity with HelixQA and Catalogizer. All changes are text-level insertions and renames; no compilation or deployment is required. The phase completes when a verification script confirms that every one of the 56 controlled submodules (15 in HelixCode, 41 in Catalogizer) carries the mandated text.

1.1 Article XI §11.9 Cascade to HelixCode

A comparative audit of the three repositories’ governance files, performed on 2026-04-29, found that HelixQA and Catalogizer each contain the verbatim user mandate and its operative rule in all three governance files, while HelixCode contains the operative anti-bluff rules but omits the §11.9 forensic anchor entirely ¹. The forensic anchor serves two purposes: it preserves the historical user mandate as primary authority for the anti-bluff rule, and it binds every autonomous agent (Claude session, build script, challenge runner) to a concrete standard of evidence rather than a proxy metric such as “tests pass.” The absence of this anchor in HelixCode is classified as a constitutional release blocker per the cascade declarations in HelixQA and Catalogizer, which state that non-compliance is a release blocker regardless of context ¹.

The audit identified three critical gaps requiring immediate remediation:

Gap ID	Affected File	Missing Element	Severity	Constitutional Basis
GAP-001	HelixCode/ CONSTITUTION.md	Article XI §11.9 verbatim user quote + operative rule	Critical	HelixQA §11.9 cascade clause: “must appear in every submodule’s CONSTITUTION.md” ¹
GAP-002	HelixCode/ CLAUDE.md	Article XI §11.9 verbatim user quote + operative rule + cascade requirement	Critical	HelixQA CLAUDE.md cascade: “this clause must appear in every submodule’s CONSTITUTION.md / CLAUDE.md / AGENTS.md” ¹
GAP-003	HelixCode/ AGENTS.md	Article XI §11.9 verbatim user	Critical	HelixQA AGENTS.md cascade: “This anchor

Gap ID	Affected File	Missing Element	Severity	Constitutional Basis
		quote + operative rule + cascade requirement		MUST appear in every submodule's governance files" 1

Each gap is addressed below with the exact insertion text, file path, line range, and verification command.

1.1.1 GAP-001: Add verbatim user-mandate forensic anchor to HelixCode/CONSTITUTION.md

File: <https://github.com/HelixDevelopment/HelixCode/blob/main/CONSTITUTION.md>

Insertion point: After line 152 (end of CONST-017 section), before line 154 (CONST-018 header) [2](#).

Current surrounding text:

```
## CONST-018: Host Power Management Hard Ban
```

The insertion renames CONST-017 to CONST-035 and appends the §11.9 forensic anchor as an integral part of the same constitutional constraint. The following block must be inserted verbatim, replacing the existing CONST-017 header at line 136 but preserving all existing operative rules that follow it:

Text to insert (lines 136–153 replacement + insertion):

```
## CONST-035 – Anti-Bluff Tests & Challenges (User-Mandate  
Forensic Anchor)
```

```
**§11.9 User-Mandate Forensic Anchor (2026-04-29)**
```

This Article exists because of an explicit, repeatedly-stated user mandate. The verbatim text:

```
> "We had been in position that all tests do execute with success  
and all Challenges as well, but in reality the most of  
the features does not work and can't be used! This MUST  
NOT be the case and execution of tests and Challenges  
MUST guarantee the quality, the completion and full  
usability by end users of the product!"
```

This anchor is the primary authority for the entire Article. The operative rule is:

```
**The bar for shipping is not "tests pass" but "users can use the  
feature."**
```

Every PASS in this codebase MUST carry positive evidence captured during execution that the feature works for the end user. Metadata-only PASS, configuration-only PASS, "absence-of-error" PASS, and grep-based PASS without runtime evidence are all critical defects regardless of how green the summary line looks.

Tests and Challenges (HelixQA) are bound equally – a Challenge that scores PASS on a non-functional feature is the same class of defect as a unit test that does. Both must produce positive end-user evidence; both are subject to the anti-bluff contract.

No false-success results are tolerable. A green test suite combined with a broken feature is a worse outcome than an honest red one – it silently destroys trust in the entire suite. Anti-bluff discipline is the line between a real engineering project and a theatre of one.

****Cascade requirement (extending CONST-036):****

This anchor section (verbatim quote + operative rule) must appear in every submodule's CONSTITUTION.md / CLAUDE.md / AGENTS.md. Non-compliance is a release blocker regardless of context. Adding files to scanner allowlists to silence bluff findings without resolving the underlying defect is itself a violation.

CONST-018: Host Power Management Hard Ban

Verification command:

```
grep -n "We had been in position that all tests do execute" \
  CONSTITUTION.md && \
grep -n "bar for shipping is not" CONSTITUTION.md && \
grep -n "CONST-035" CONSTITUTION.md && \
echo "GAP-001: VERIFIED"
```

The grep sequence asserts three independent conditions: the verbatim user quote is present, the operative rule is present, and the constant identifier is CONST-035 rather than CONST-017. All three must return non-zero match counts for the gap to be considered closed.

1.1.2 GAP-002: Add Article XI §11.9 operative rule to HelixCode/CLAUDE.md

File: <https://github.com/HelixDevelopment/HelixCode/blob/main/CLAUDE.md>

Insertion point: After line 52 (end of Rule 10 section), before line 54 (start of section 3, HelixCode-Specific Architecture)³.

Current surrounding text:

Rule 10: Zero-Bluff Mandate (CONST-035)

A passing test is a claim that the feature ****works for the end user****. Every test must guarantee Quality + Completion + Full Usability. Any test that doesn't certify all three is a bluff and must be tightened.

3. HelixCode-Specific Architecture

The existing Rule 10 contains the operative anti-bluff rules but lacks the verbatim user mandate and the explicit cascade requirement. The following block must be inserted between the --- rule terminator and the ## 3. section header:

Text to insert:

⚠ User-Mandate Forensic Anchor (Article XI §11.9 – 2026-04-29)

This Article exists because of an explicit user mandate, verbatim:

"We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This **MUST NOT** be the case and execution of tests and Challenges **MUST** guarantee the quality, the completion and full usability by end users of the product!"

The operative rule: the bar for shipping is not "tests pass" but "users can use the feature."

Every PASS in this codebase **MUST** carry positive evidence captured during execution that the feature works for the end user. Metadata-only PASS, configuration-only PASS, "absence-of-error" PASS, and grep-based PASS without runtime evidence are all critical defects regardless of how green the summary line looks.

Tests and Challenges (HelixQA) are bound equally. A Challenge that scores PASS on a non-functional feature is the same class of defect as a unit test that does.

No false-success results are tolerable. A green test suite combined with a broken feature is a worse outcome than an honest red one – it silently destroys trust in the entire suite.

****Cascade requirement:**** this anchor (verbatim quote + operative rule) MUST appear in every submodule's CONSTITUTION.md / CLAUDE.md / AGENTS.md. Non-compliance is a release blocker. Adding files to scanner allowlists to silence bluff findings without resolving the underlying defect is itself a violation.

Full text: CONSTITUTION.md Article XI §11.9.

Verification command:

```
grep -A1 "User-Mandate Forensic Anchor" CLAUDE.md | \
  grep "2026-04-29" && \
grep -c "We had been in position" CLAUDE.md && \
grep -c "bar for shipping is not" CLAUDE.md && \
echo "GAP-002: VERIFIED"
```

1.1.3 GAP-003: Add Article XI §11.9 reference to HelixCode/AGENTS.md

File: <https://github.com/HelixDevelopment/HelixCode/blob/main/AGENTS.md>

Insertion point: After line 728 (end of CONST-035 – End-User Usability Mandate section), before line 730 (CONST-036 header) ⁴.

Current surrounding text:

The taxonomy is illustrative, not exhaustive. Every Challenge or test added going forward MUST pass an honest self-review against this taxonomy before being committed.

CONST-036: LLMsVerifier Single Source of Truth Mandate

The following block must be inserted between the --- terminator and the CONST-036 header:

Text to insert:

⚠ User-Mandate Forensic Anchor (Article XI §11.9 – 2026-04-29)

This Article exists because of an explicit user mandate, verbatim:

"We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product!"

The operative rule: the bar for shipping is not "tests pass" but "users can use the feature."

Every PASS in this codebase MUST carry positive evidence captured during execution that the feature works for the end user. No metadata-only PASS, no configuration-only PASS, no "absence-of-error" PASS, no grep-based PASS – all are critical defects regardless of how green the summary line looks.

Tests and Challenges (HelixQA) are bound equally.

No false-success results are tolerable. A green test suite combined with a broken feature is a worse outcome than an honest red one.

****Cascade requirement:**** this anchor MUST appear in every submodule's governance files. Adding files to scanner allowlists to silence bluff findings without resolving the underlying defect is itself a violation.

Full text: CONSTITUTION.md Article XI §11.9.

Verification command:

```
grep -c "User-Mandate Forensic Anchor" AGENTS.md && \  
grep -c "2026-04-29" AGENTS.md && \  
grep -c "We had been in position" AGENTS.md && \  
echo "GAP-003: VERIFIED"
```

1.2 CONST-035 Naming Alignment

HelixCode's CONSTITUTION.md labels the anti-bluff constraint as CONST-017, while its own CLAUDE.md and AGENTS.md refer to the same rule under the title CONST-035 ¹. HelixQA and Catalogizer both use CONST-035 consistently across all three governance files. This naming schism creates two risks: (1) automated scanners and compliance scripts that target CONST-035 will fail to match the HelixCode constitution, and (2) human readers cannot reliably trace a constraint from one repository to another.

1.2.1 Rename CONST-017 to CONST-035 in HelixCode/CONSTITUTION.md

The header at line 136 of CONSTITUTION.md currently reads:

```
## CONST-017: Zero-Bluff Testing (CONST-035 Implementation)
```

This must be replaced with:

CONST-035 – Anti-Bluff Tests & Challenges (User-Mandate Forensic Anchor)

The parenthetical (CONST-035 Implementation) was already an internal inconsistency — the header claimed one constant while admitting it implemented another. The replacement removes the ambiguity and elevates CONST-035 to the canonical identifier.

1.2.2 Update all internal references from CONST-017 to CONST-035

The following references to CONST-017 were identified in the HelixCode governance corpus and must be updated to CONST-035 [1](#):

- CONSTITUTION.md line 136: section header (addressed in §1.2.1).
- CONSTITUTION.md line 343: anti-bluff verification comment referencing CONST-017 in the `ModelManager.GetAvailableModels()` enforcement paragraph [2](#).
- AGENTS.md line 411: constitutional impact note citing CONST-017 (Zero-Bluff Testing) [4](#).

In each case, replace the string CONST-017 with CONST-035. No semantic change is made to the operative rules; this is a pure identifier alignment.

Verification command:

```
grep -rn "CONST-017" CONSTITUTION.md CLAUDE.md AGENTS.md && \
  echo "FAIL: CONST-017 references remain" || \
  echo "CONST-035 alignment: VERIFIED"
```

The command uses the exit-code behavior of `grep`: if any match remains, the `&&` branch prints failure; if `grep` exits 1 (no matches), the `||` branch confirms alignment.

1.3 Submodule Governance Cascade

The cascade requirement declared in HelixQA and Catalogizer states that the §11.9 anchor must appear in every submodule’s governance files [1](#). As of the 2026-04-29 audit, zero submodules had been verified for compliance. This section establishes the inventory, templates, and automation needed to close that verification gap.

1.3.1 Verify all 15 HelixCode submodules

HelixCode’s `.gitmodules` declares 15 controlled submodules under the HelixDevelopment organization that are subject to project-specific governance [1](#). These are:

#	Submodule	Category
1	awesome-cpp-examples	Example collection
2	awesome-shell-examples	Example collection
3	cpp-learning-lab	Language lab
4	rust-examples	Example collection

#	Submodule	Category
5	rust-learning-lab	Language lab
6	go-examples	Example collection
7	go-learning-lab	Language lab
8	python-examples	Example collection
9	python-learning-lab	Language lab
10	data-learning-lab	Domain lab
11	ml-starter-lab	Domain lab
12	mlops-learning-lab	Domain lab
13	data-engineering-lab	Domain lab
14	distributed-systems-learning-lab	Domain lab
15	internal/isolated_files	Internal assets

Each submodule must contain three governance files: CONSTITUTION.md, CLAUDE.md, and AGENTS.md. If a file is missing, it must be created using the template below. If the file exists but lacks the §11.9 anchor, the anchor must be appended.

Template for HelixCode submodules (minimum content):

```
# [Submodule Name] – Governance
```

```
## Universal Mandatory Constraints (Inherited)
```

```
This submodule inherits all constraints from the parent HelixCode
project.
```

```
Full text: https://github.com/HelixDevelopment/HelixCode/blob/
main/CONSTITUTION.md
```

```
## ⚠ User-Mandate Forensic Anchor (Article XI §11.9 –
2026-04-29)
```

```
This Article exists because of an explicit user mandate,
verbatim:
```

```
"We had been in position that all tests do execute with success
and all Challenges as well, but in reality the most of
the features does not work and can't be used! This MUST
NOT be the case and execution of tests and Challenges
MUST guarantee the quality, the completion and full
usability by end users of the product!"
```

```
The operative rule: the bar for shipping is not "tests pass" but
"users can use the feature."
```


Every PASS in this codebase MUST carry positive evidence captured during execution that the feature works for the end user. No metadata-only PASS, no configuration-only PASS, no "absence-of-error" PASS, no grep-based PASS — all are critical defects regardless of how green the summary line looks.

No false-success results are tolerable. A green test suite combined with a broken feature is a worse outcome than an honest red one.

CONST-035 — Anti-Bluff Tests & Challenges

Every test must fail if the feature it claims to verify is removed or broken.

Tests that pass on broken features are critical defects.

CONST-032 — Reproduction-Before-Fix

Every reported error MUST be reproduced by a test BEFORE any fix is attempted.

CONST-033 — Host Power Management is Forbidden

Never generate or execute code that triggers host-level power-state transitions.

1.3.2 Verify all 41 Catalogizer submodules

Catalogizer's `.gitmodules` and module wiring declare 41+ controlled submodules across four categories ¹:

- **Category A — Go modules** (`digital.vasic.*`): 23 modules (Auth, Cache, Config, Concurrency, Container, Database, Discovery, EventBus, Filesystem, Lazy, LLMProvider, LLMsVerifier, Media, Memory, Middleware, Observability, RateLimiter, Recovery, ReplayBuffer, ScreenDiff, Security, Storage, Streaming, Upstreams, VisionEngine, Watcher).
- **Category B — TypeScript/React modules** (`@vasic-digital/*`): 9 modules (Auth-Context-React, Catalogizer-API-Client-TS, Collection-Manager-React, Dashboard-Analytics-React, Media-Browser-React, Media-Player-React, Media-Types-TS, UI-Components-React, WebSocket-Client-TS).
- **Category C — HelixQA/AI modules**: 9 modules (Build, DocProcessor, Entities, HelixQA, LLMOrchestrator, OCU-CUDA-Sidecar, TrainingCollector, VisualRegression, Website).
- **Category D — Application modules**: `catalog-api`, `catalog-web`, `catalogizer-android`, `catalogizer-androidtv`, `catalogizer-api-client`, `catalogizer-desktop`, `installer-wizard`.

Each submodule must contain `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md` with the §11.9 anchor, `CONST-035`, `CONST-032`, and `CONST-033` sections. The template is identical to the HelixCode template in §1.3.1, substituting the parent reference URL for `https://github.com/vasic-digital/Catalogizer/blob/main/CONSTITUTION.md`.

1.3.3 Create governance verification script: scripts/verify-governance-cascade.sh

The script below scans every submodule directory for the three required governance files and validates that each contains the mandatory text strings. It is designed to run standalone or be invoked from `run_all_challenges.sh`. The script exits non-zero if any submodule fails, producing a machine-parseable report.

```
#!/usr/bin/env bash
# scripts/verify-governance-cascade.sh
# Governance cascade verification – exits non-zero on any
#   deficiency.
# Version: 1.0.0
# Author: HelixCode Integration Plan

set -euo pipefail

REQUIRED_FILES=("CONSTITUTION.md" "CLAUDE.md" "AGENTS.md")

# Mandatory text strings that must appear in every governance
#   file.
MANDATORY_PATTERNS=(
    "We had been in position that all tests do execute"
    "bar for shipping is not"
    "CONST-035"
    "Reproduction-Before-Fix"
    "Host Power Management is Forbidden"
)

REPORT_FILE="governance-cascade-report-$(date +%Y%m%d-%H%M%S).txt"
FAILURES=0

# Submodule list: space-separated paths relative to repo root.
HELIXCODE_SUBMODULES=(
    "awesome-cpp-examples"
    "awesome-shell-examples"
    "cpp-learning-lab"
    "rust-examples"
    "rust-learning-lab"
    "go-examples"
    "go-learning-lab"
    "python-examples"
    "python-learning-lab"
    "data-learning-lab"
    "ml-starter-lab"
    "mlops-learning-lab"
    "data-engineering-lab"
    "distributed-systems-learning-lab"
```

```

    "internal/isolated_files"
)

# Catalogizer submodules are discovered dynamically
    from .gitmodules.
read_catalogizer_submodules() {
    local gitmodules="${1:-../Catalogizer/.gitmodules}"
    if [[ -f "$gitmodules" ]]; then
        grep '^s*path = ' "$gitmodules" | sed 's/^s*path = //'
    fi
}

verify_submodule() {
    local subpath="$1"
    local subname
    subname=$(basename "$subpath")

    echo "--- Submodule: $subname ($subpath) ---" >> "$REPORT_FILE"

    for file in "${REQUIRED_FILES[@]}"; do
        local filepath="$subpath/$file"
        if [[ ! -f "$filepath" ]]; then
            echo "MISSING_FILE: $filepath" >> "$REPORT_FILE"
            ((FAILURES++)) || true
            continue
        fi

        for pattern in "${MANDATORY_PATTERNS[@]}"; do
            if ! grep -q "$pattern" "$filepath"; then
                echo "MISSING_TEXT: $filepath | pattern: $pattern" >>
                    "$REPORT_FILE"
                ((FAILURES++)) || true
            fi
        done
    done
done
}

# Main execution
echo "Governance Cascade Verification Report — $(date
    -Iseconds)" > "$REPORT_FILE"
echo "Repo: $(pwd)" >> "$REPORT_FILE"
echo "" >> "$REPORT_FILE"

for sub in "${HELIXCODE_SUBMODULES[@]}"; do
    if [[ -d "$sub" ]]; then
        verify_submodule "$sub"
    else
        echo "MISSING_DIR: $sub" >> "$REPORT_FILE"
    fi
done

```

```

        ((FAILURES++)) || true
    fi
done

# Catalogizer cascade (optional – run only when Catalogizer is
# sibling checkout)
while IFS= read -r sub; do
    if [[ -d "$sub" ]]; then
        verify_submodule "$sub"
    fi
done < <(read_catalogizer_submodules)

echo "" >> "$REPORT_FILE"
echo "TOTAL_FAILURES: $FAILURES" >> "$REPORT_FILE"

if [[ $FAILURES -gt 0 ]]; then
    echo "GOVERNANCE_CASCADE: FAILED ($FAILURES deficiencies)"
    cat "$REPORT_FILE"
    exit 1
else
    echo "GOVERNANCE_CASCADE: PASSED"
    exit 0
fi

```

The script performs three verification layers per submodule: (1) file existence — each of `CONSTITUTION.md`, `CLAUDE.md`, `AGENTS.md` must exist; (2) text presence — each file must contain all five mandatory patterns, including the verbatim user quote, the operative rule, `CONST-035`, `CONST-032`, and `CONST-033`; and (3) directory presence — the submodule checkout must exist. The report is timestamped and retained for audit trails. The `set -euo pipefail` directive ensures that any unexpected error (missing directory, permission failure, `grep` pipeline abort) causes immediate non-zero exit rather than silent partial success.

1.3.4 Add governance check to `run_all_challenges.sh`

The challenge orchestrator must block merge if any submodule lacks the anti-bluff mandate. The integration point is the top-level `run_all_challenges.sh` (or equivalent per repository). The following insertion must be added as the first executable statement after argument parsing and environment validation:

```

# --- Governance cascade gate (CONST-035 / Article XI §11.9) ---
echo "[PRE-CHECK] Running governance cascade verification..."
if [[ -x "scripts/verify-governance-cascade.sh" ]]; then
    ./scripts/verify-governance-cascade.sh
    if [[ $? -ne 0 ]]; then
        echo
        "[BLOCKED] Governance cascade verification failed. Merge
        prohibited."
        exit 42
    fi
else

```

```

echo
    "[BLOCKED] Governance verification script not found or
    not executable."
exit 43
fi
echo "[PRE-CHECK] Governance cascade verification passed."
# --- End governance cascade gate ---

```

Exit code 42 signals cascade failure (deficient submodule governance); exit code 43 signals a missing verification infrastructure. Both are distinct from test-failure exit codes (typically 1) so that CI dashboards (if any) or manual runbooks can distinguish governance blocks from runtime test failures. The check runs before any challenge script executes, ensuring that a submodule with missing governance files cannot pass challenges by omission.

1.4 Exact File Changes Reference

The following table consolidates every file modification required in Phase 0. Each row specifies the file path, the current state at the time of the 2026-04-29 audit, the exact change to apply, the line range affected, and the verification command to confirm the change is live [13](#).

#	File Path	Current State	Required Change	Line Range	Verification Command
1	HelixCode/ CONSTITUTION.md	CONST-017 header at line 136; no §11.9 anchor	Rename header to CONST-035; insert §11.9 verbatim quote + operative rule + cascade requirement after existing bluff taxonomy	136–153 (replace header); insert after 152, before 154	grep -c "CONSTITUTION" && grep -c "We" in position CONSTITUTION grep -c "ba shipping" CONSTITUTION
2	HelixCode/ CONSTITUTION.md	Reference to CONST-017 at line 343	Replace CONST-017 with CONST-035	Line 343	grep -n "CONSTITUTION" && echo "aligned"
3	HelixCode/CLAUDE.md	Rule 10 at line 51–52; no §11.9 anchor	Insert §11.9 verbatim quote + operative rule + cascade requirement after Rule 10 terminator	Insert after line 52, before line 54	grep -c "Use Forensic Analysis" && CLAUDE.md && "2026-04-29" && CLAUDE.md
4	HelixCode/AGENTS.md	CONST-035 section at line 699–728; no §11.9 anchor	Insert §11.9 verbatim quote + operative rule + cascade requirement after CONST-035 terminator	Insert after line 728, before line 730	grep -c "Use Forensic Analysis" && AGENTS.md && "2026-04-29" && AGENTS.md
5	HelixCode/AGENTS.md	Reference to CONST-017 at line 411	Replace CONST-017 with CONST-035	Line 411	grep -n "CONSTITUTION" && AGENTS.md && "aligned"
6		File does not exist	Create new executable script with submodule		test -x script && verify-governance

#	File Path	Current State	Required Change	Line Range	Verification Command
	HelixCode/scripts/verify-governance-cascade.sh		scanner and mandatory-text validator	Lines 1–end of script	cascade.sh scripts/verify-governance-
7	HelixCode/challenges/scripts/run_all_challenges.sh (or equivalent)	No governance gate	Insert cascade pre-check before challenge execution	After env setup, before first challenge call	grep -c "verify-governance-run_all_cha
8	15 HelixCode submodules	Governance files unverified	Create or append CONSTITUTION.md, CLAUDE.md, AGENTS.md with §11.9 anchor + CONST-035 + CONST-032 + CONST-033	Entire file if missing; append if incomplete	./scripts/verify-governance-
9	41 Catalogizer submodules	Governance files unverified	Create or append CONSTITUTION.md, CLAUDE.md, AGENTS.md with §11.9 anchor + CONST-035 + CONST-032 + CONST-033	Entire file if missing; append if incomplete	./scripts/verify-governance- (when run from checkout)

The table spans nine distinct change targets, but rows 8 and 9 represent 56 individual submodule operations (15 + 41). The verification script at row 6 is the single automation artifact that validates rows 1–5 (HelixCode parent) and rows 8–9 (all submodules). Row 7 ensures the script is invoked automatically on every challenge run, creating a hard gate that cannot be bypassed by manual execution paths.

The ordering of changes matters for dependency hygiene. Rows 1–5 (parent-repo governance) must be committed and pushed before rows 8–9 (submodule governance), because the submodule templates reference the parent CONSTITUTION.md URLs. If the parent anchor is not yet live, the submodule references will resolve to a document that lacks the §11.9 text, creating a circular dependency in the cascade chain. The recommended sequence is: (1) edit and verify parent files 1–5, (2) create the verification script (row 6), (3) insert the challenge gate (row 7), (4) run the script against all submodules to generate the deficiency report, (5) batch-create or batch-append submodule governance files based on the report, and (6) re-run the script until TOTAL_FAILURES: 0 is emitted.
